

NOTE:

- These are not exact final questions. These questions are intended to review the material.

1 Quick Questions

- (a) $f(n) + O(f(n)) =$ _____.
- (b) If $f(n) = \Theta(n \log n)$, then $f\left(\frac{n}{2}\right) =$ _____ $(f(n))$.
- (c) Although Merge-Sort's worst-case runtime is asymptotically equivalent to Quick-Sort's average-case runtime, Merge-Sort is rarely used in practice since _____ and _____.
- (d) If $T(n) =$ _____ $T\left(\frac{n}{3}\right) + 50n^2 + 25n$, then $T(n) = \Theta(n^2 \log n)$.
- (e) Consider array A that contains all integers from 1 to 1000 in some arbitrary order. We sort this array using a modified version of Randomized-Quick-Sort, which sets the pivot to the median of 3 randomly picked elements in each partition. This algorithm will compare 1 and 1000 during partitioning with probability _____.

2 Depth-First Search

Assume that we have run Depth-First Search (DFS) on a directed graph $G(V, E)$ and obtained the following discovery and finishing times for each vertex in V :

Vertex	d	f
v_1	1	10
v_2	2	3
v_3	4	9
v_4	5	6
v_5	7	8
v_6	11	12
v_7	13	16
v_8	14	15

Answer the following questions assuming that we do not know E . You do not need to justify your answers, but justification may help in receiving partial credit.

- (a) (4 points) List all edges that must exist.
- (b) (4 points) List all possible edges of v_4 (both incoming and outgoing), which, if they existed, would be classified as cross edges according to this DFS run.
- (c) (4 points) Assuming that there are no cycles in the graph, list all vertices that cannot possibly have the largest finishing time in any DFS run.
- (d) (4 points) List all vertices that *cannot* be in the same strongly connected component as v_3 .

3 Presentation Question: Graph A

NOTE: For graph algorithms you need to provide following items:

- Representation: How you construct the graph?
- Pseudocode
- Proof of correctness
- Run time analysis

NOTE: Unlike the recitation presentation problems, and the problem shown here, the exam problems may be framed more abstractly.

Problem Statement: You are planning the seating arrangement for a wedding given a list of guests, V . Suppose you are also given a lookup table T where $T[u]$ for $u \in V$ is a list of guests that u knows. If u knows v , then v knows u . You are required to arrange the seating such that any guest at a table knows every other guest sitting at the same table either directly or through some other guests sitting at the same table. For example, if x knows y , and y knows z , then x, y, z can sit at the same table. Describe an efficient algorithm that, given V and T , returns the minimum number of tables needed to achieve this requirement. Analyze the running time of your algorithm.

Example

Input:

$V = \{1, 2, 3, 4, 5, 6\}$

$T[1] = \{2, 3\}$ $T[2] = \{1, 3\}$ $T[3] = \{1, 2\}$

$T[4] = \{5, 6\}$ $T[5] = \{4, 6\}$ $T[6] = \{4, 5\}$

Output:

Minimum number of tables: 2

Explanation:

The graph of guest connections can be visualized as two connected components:

- Component 1: $\{1, 2, 3\}$ (everyone knows each other within this group).
- Component 2: $\{4, 5, 6\}$ (everyone knows each other within this group).

4 Presentation Question: Dynamic Programming

NOTE: For dynamic programming problems you need to provide following items:

- Recursive Formulation
- Optimum Substructure Proof
- **Pseudocode (Solution Reconstruction is NOT Needed)**
- Run time analysis

NOTE: Unlike the recitation presentation problems, and the problem shown here, the exam problems may be framed more abstractly.

Problem Statement: You are programming an automated system to extract the valuable isotope uranium-310, which is only formed in the fuel rods of nuclear reactors. The extraction system uses a pair of robot arms to hold a single fuel rod that is divided into n sections. Each section k has a known U^{310} concentration v_k (not all of the values are equal).

One arm holds the rod above the reactor, and the other has a cutting tool that can remove a single section at a time for processing. This means that if section k is removed, then the arm can only hold onto part of the rod containing sections greater than, or less than, k . The other sections fall back into the reactor and are melted down. For example, if the rod has 20 sections at the start, and you harvest Section 5, you must discard either Sections 1-4 (leaving you able to process any section between 6 and 20), or Sections 6-20 (leaving you only able to process any section between 1 and 4). Assuming you discarded Sections 1-4 and next process Section 11, you will have to discard either sections 6-10, or Sections 12-20. This repeats until there is only 1 section of the fuel rod is left, which will be the last section that you process.

Furthermore, since U^{310} is a radioactive element, it decays exponentially over time. Specifically, if Section k is harvested at step t , the total yield y_k will be $v_k/(1 + \alpha)^{t-1}$. This is why the optimal solution is *not necessarily* to process the whole rod (for instance by starting at Section 1 and working forward, discarding nothing). The constant α is the same for all the sections.

Devise an algorithm to determine the maximum possible U^{310} yield attainable given v and α . You do not need to return the sequence of operations which will be performed. (Prove the algorithm is correct by proving the optimal substructure of the problem, and state and justify its running time.)

Example

Input:

$n = 4, \alpha = 0.2, v = [10, 40, 30, 50]$

Output: 114.19

Generated by harvesting Sections $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ to produce expected yield

$$50 + \frac{30}{1.2} + \frac{40}{1.2^2} + \frac{10}{1.2^3} \approx 114.19$$